Deep Learning and Computation

The deep learning story — Lecture 18 (Chapter 10, Part 1)

Amir Farbin

Part 1 — The deep learning story

Where we left off

- Classical machine learning learns **where to draw the boundary**, on features someone chose
- Deep learning learns **the features too**: from pixels, sound or particle hits to an answer
- A **feature** is a number that describes an example (a pixel's brightness, a momentum, M_{TR}^2); a **model** maps features to an answer
- Today: a first, intuitive picture of what these networks do. The pieces come later

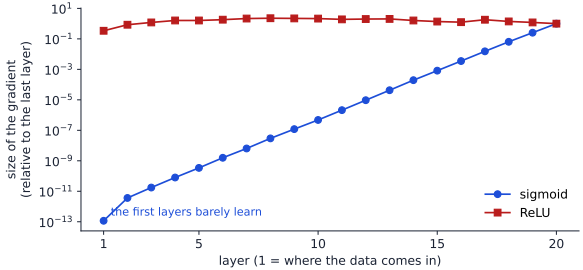
An old idea

- **1943:** a mathematical **neuron** (or **unit**): multiply each input by a **weight**, add them up, and fire past a threshold
- **1958:** the perceptron, a machine that **learned** its weights, with motor-turned resistors
- **1989: universal approximation**: one **hidden layer** (a layer between input and output) can approximate **any** continuous function
  - but it says nothing about **how many** neurons, or how to **find** the weights

Deeper is more efficient

- Universality says one wide layer is **enough**. It doesn't say it's **efficient**
- A **deep** network (many hidden layers) represents many functions with **far fewer** neurons than a **shallow** one (one hidden layer)
- Like computing in steps: each layer reuses what the one before it computed

But deep networks wouldn't train



- The gradient for an early layer is a **product** of one factor per layer above it (the chain rule)
- With sigmoids each factor is small, so the **first layers**, where the data comes in, barely learn: the **vanishing gradient**
- *Terms:* the **gradient** says how much the error changes when each weight changes; the **sigmoid** is an S-shaped activation between 0 and 1

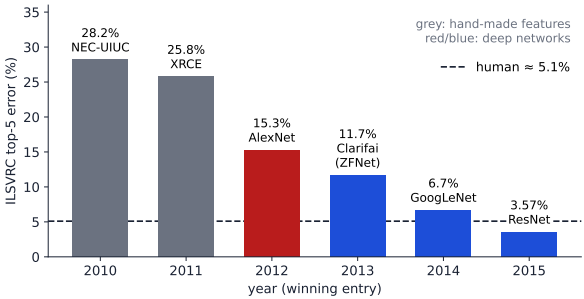
The quiet years

- **1969:** a book proves the limits of a single layer. Interest and funding dry up
- **1991:** the vanishing gradient is identified. Deep networks are set aside
- The 1990s: AI turns to **symbolic** systems, logic and rules, which don't get very far either
- A few keep going: LeCun's networks read zip codes, then cheques

What changed

- **A simpler activation: ReLU**, $\max(0, z)$: "a line and a line". Its slope is 1 wherever it's on, so gradients don't shrink. (The **activation function** is the nonlinearity a neuron applies to its sum)
- **Better training: stochastic gradient descent** (update the weights from a small random **mini-batch** of examples at a time), better starting weights, normalization, shortcut connections
- **Big data:** the internet; millions of labeled images
- **GPUs** (graphics processing units): built for games, they do exactly the arithmetic training needs

2012: the renaissance begins

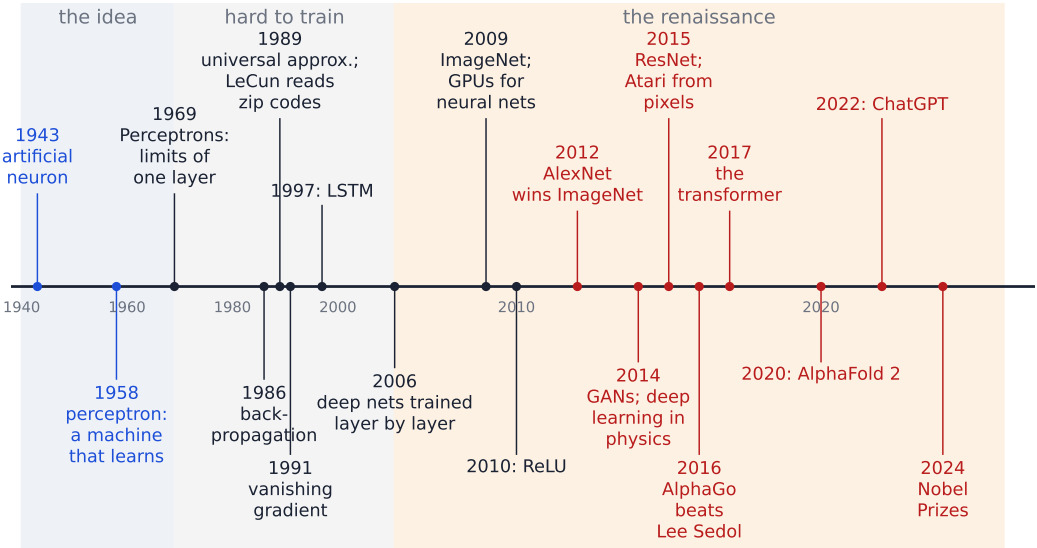


- **ImageNet:** about a million labeled images, 1,000 categories. Labels by paid crowd workers
- Hand-built features: ~26% error. **AlexNet**, a deep network on two gaming GPUs: 15%. By 2015, better than a person
- **Top-5 error:** how often the right answer is *not* among the network's five best guesses

And then, everything

- **2013–15:** Atari games learned from the pixels and the score alone
- **2014:** deep networks in particle physics, on the SUSY dataset you've used
- **2016:** AlphaGo beats a world champion at Go; players start learning from it
- **2020:** AlphaFold predicts protein structures
- **2022:** ChatGPT
- **2024:** Nobel Prizes in physics (Hopfield, Hinton) and chemistry (Hassabis, Jumper)

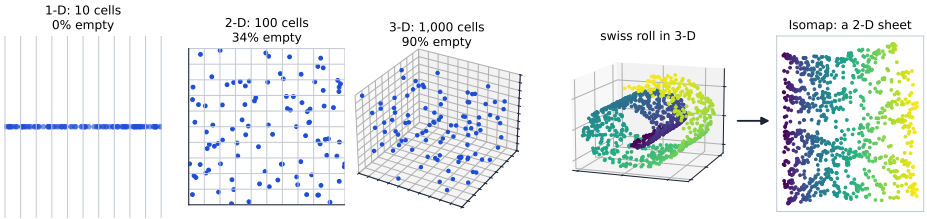
The timeline



Learning a representation

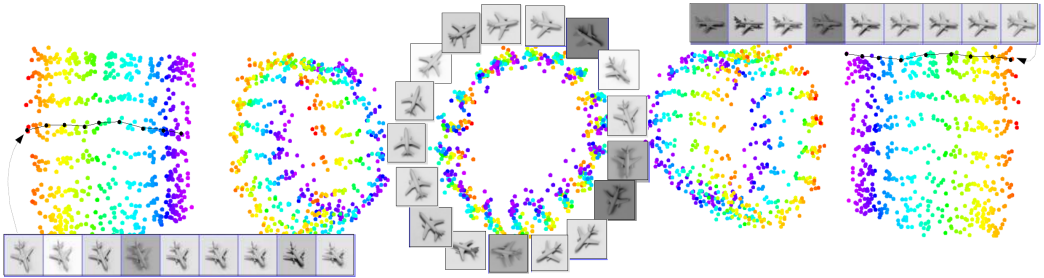
- Your brain keeps a **model** of this room from glimpses, and acts on the model
- A network maps raw data into a **new space** where the task is easy. The new numbers are a **representation** of the data
- The perfect function exists, because brains compute it. We **approximate** it from examples

Most of space is empty, and data lives on a thin sheet



- More dimensions, more boxes: almost all of them empty (the **curse of dimensionality**)
- A 100×100 photo is one point in 10,000 dimensions, but all photos of one face lie on a thin, curved **sheet**. Walking on it is **turning your head**, or **smiling**
- That sheet is a **manifold**: a low-dimensional surface inside a high-dimensional space. Learning features is learning its **coordinates**

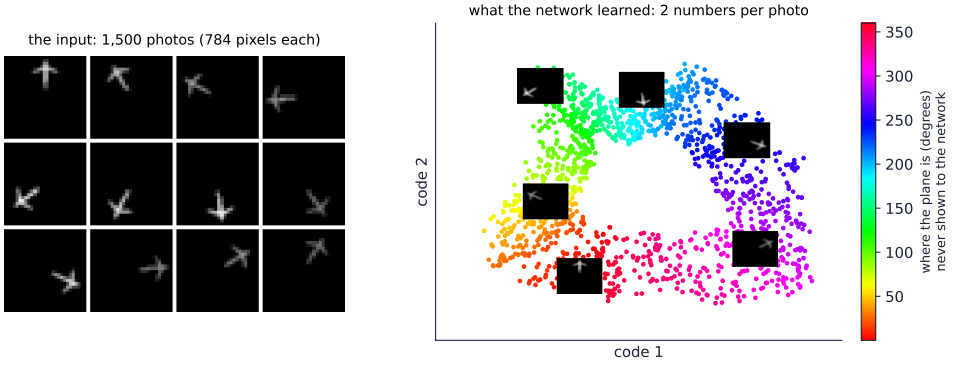
A network can learn the sheet



- Photos of a toy plane, turned and lit from many directions, mapped by a **trained network** to 3 numbers: **dimensionality reduction**
- Told only which photos are neighbours, it finds a **cylinder**: around it the plane turns; along it the light moves

Hadsell, Chopra & LeCun, "Dimensionality reduction by learning an invariant mapping", CVPR 2006

Try it: a plane flying in circles

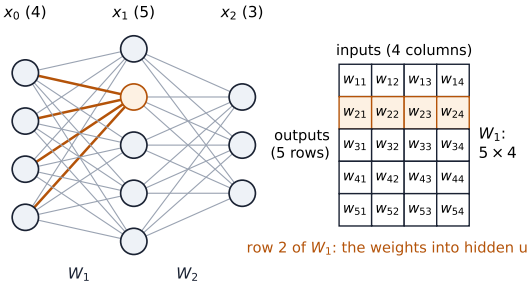


- 1,500 photos of 784 pixels; an **autoencoder** (a network trained to reproduce its input) squeezes each to **two numbers**, its **code**
- It was never told where the plane is, but the codes form a **ring** in the right order. Dimmer photos sit further in

Why deep: build on what's below

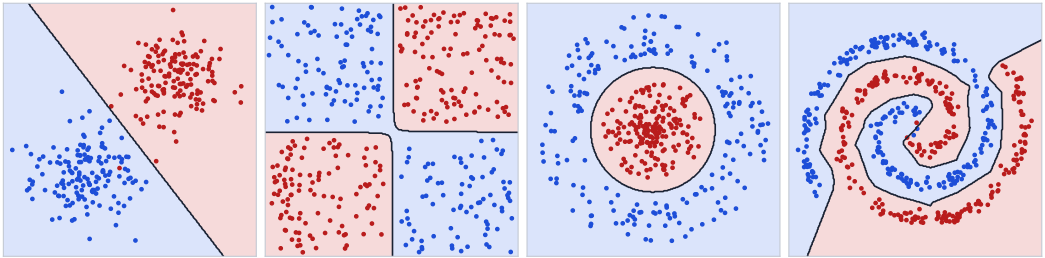
- Pixels $\rightarrow$ edges $\rightarrow$ parts $\rightarrow$ objects. Characters $\rightarrow$ words $\rightarrow$ sentences
- One layer **builds** a feature, the next **combines** it
- Low layers are **shared** by everything; high layers are **specific**: a **feature hierarchy**

The building block



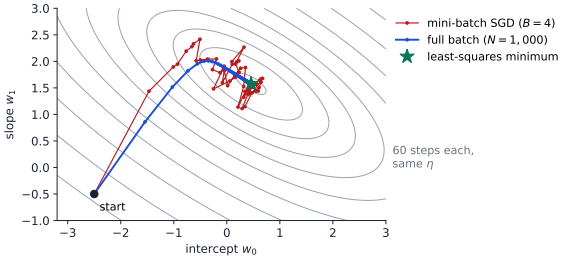
- A **layer**: a linear model $(Wx + b)$, then a nonlinearity. A **network**: layers stacked
- The **weights** W and **biases** b are the **parameters**: the numbers training adjusts
- Without the nonlinearity, a stack of layers is still **one** linear model

Feature engineering, or feature learning



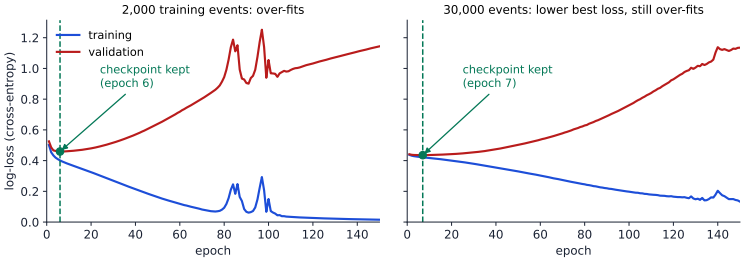
- XOR: multiply the two inputs and one neuron is enough. The circle: all you need is the **radius**
- The spiral: let a network **find** the features
- **Feature engineering:** a person designs the features. **Feature learning:** the network finds them

Training is walking downhill



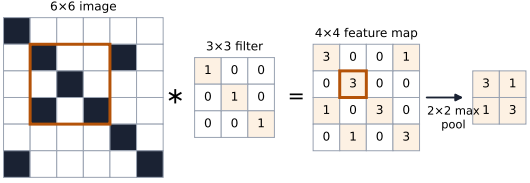
- Height = how wrong you are, the **cost** (or **loss**) function. Position = the parameters. Step **against** the slope: **gradient descent**
- Estimate the slope on a **mini-batch**: many cheap, noisy steps, and the noise helps
- The step size is the **learning rate**. One pass through all the training data is an **epoch**

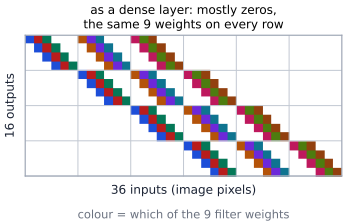
It can memorize



- A big enough network can learn every training example by heart: **overfitting**. It then fails on new data: it doesn't **generalize**
- Keep a **validation set** it never trains on, and watch the two **learning curves** (error per epoch) separate
- **Dropout**: knock out random units while training, so it can't lean on any one

A filter is a detector



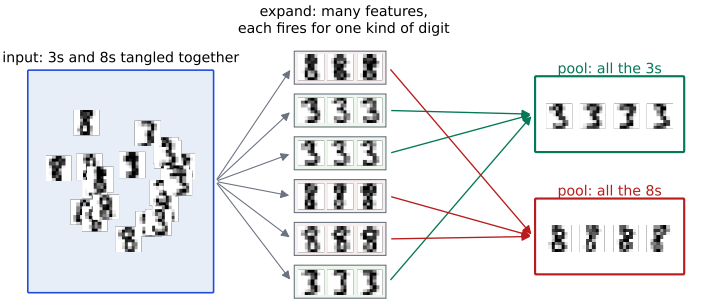


- Slide a small patch, a **filter** (or **kernel**), over the image, multiply and add: a big number **where the pattern is**. This is a **convolution**
- The identity-matrix patch lights up on **diagonal lines**: a diagonal-line detector
- One filter per pattern gives one **feature map** per pattern

Stack, shrink, repeat

- Shrink the maps by **pooling** (keep the largest value in each small block), then look for patterns **in the maps**: lines make curves, curves make parts
- Repeat until the image is one pixel wide, with many features
- Local and general $\rightarrow$ **global and specific**: the chair detector doesn't say where the chair is
- I don't build the detectors. I set it up as "cat or not", and it **learns** every filter. A network built this way is a **convolutional neural network (CNN)**

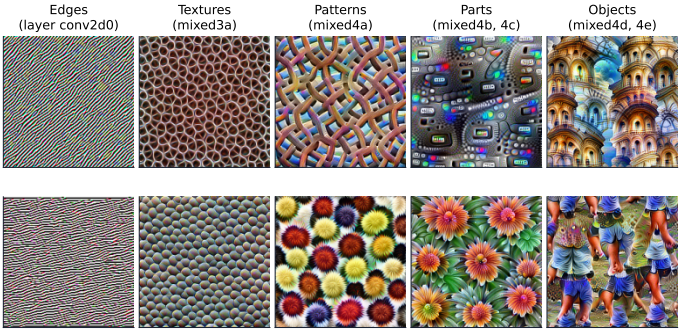
Expand, then pool



in the expanded space a simple cut separates them; pooling keeps what matters and forgets the style

- Lift the data into **more** dimensions, where a simple cut separates it; then **pool** what's alike

What the layers learn

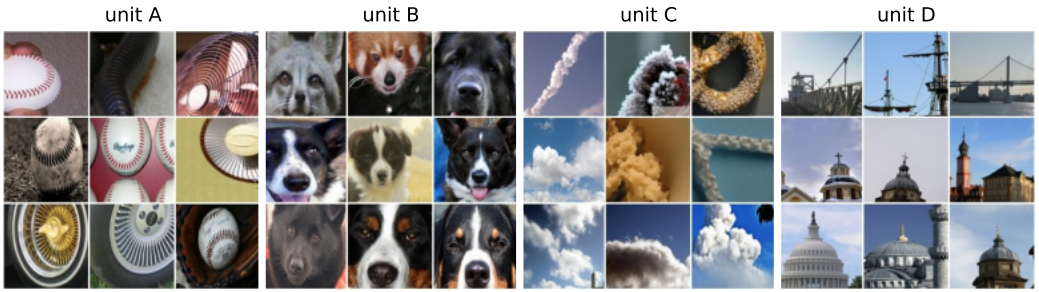


early layers → late layers: what one unit in each layer responds to most strongly (optimized images)

- Early layers: **edges and textures**, the same whatever you train on. Later: parts, then objects

Olah, Mordvintsev & Schubert, "Feature Visualization", Distill (2017), CC-BY 4.0

Which photos light up a unit

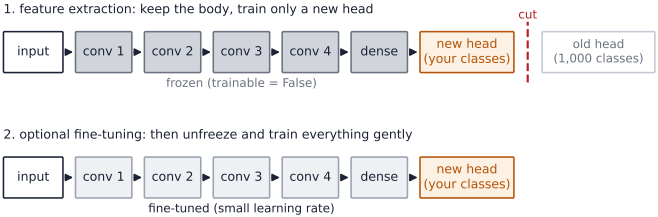


for each of four units deep in the network: the 9 photos that activate it most

- Run many photos through, and keep the ones that **activate** one unit most (its **activation** is the number it outputs)
- Deep in the network, units have become detectors of **things**: dog faces, clouds, domes

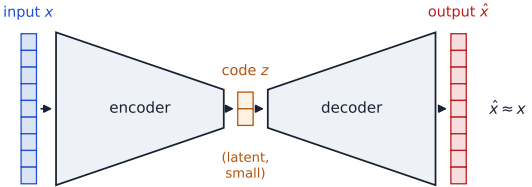
Olah, Mordvintsev & Schubert, "Feature Visualization", Distill (2017), CC-BY 4.0

Features belong to the domain



- **Transfer learning:** start from a network already trained on a big dataset for one task, and reuse it for a new task with much less data
- What a network learns about images is about **images**, not just its task
- Replace its last layer, the **head**, with one for your task and train only that; then **fine-tune**: train all layers a little, with a small learning rate. Downloaded networks are **pre-trained models**

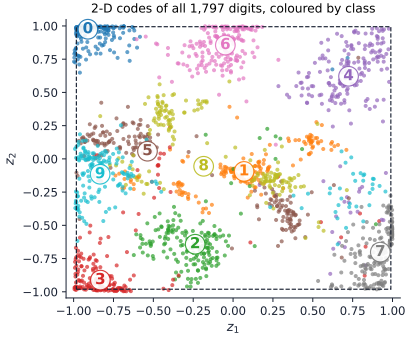
Learning without labels: the autoencoder

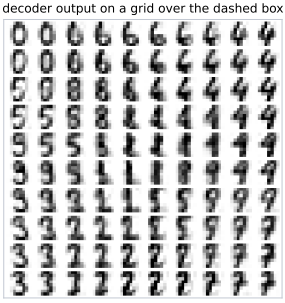


loss $= \|x - \hat{x}\|^2$ (the target is the input itself: no labels, self-supervised)

- An **encoder** squeezes the input through a **bottleneck**, a **decoder** reproduces it. No **labels** (known answers) needed: **unsupervised** (or **self-supervised**) learning
- The small **code** in the middle, the **latent space**, keeps what's unique about each example
- Change the target: noisy in, clean out; small in, large out

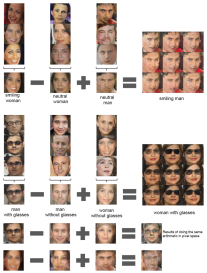
The latent space is a map

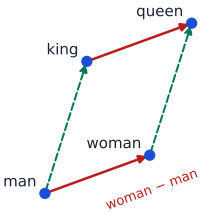




- Squeeze digits to **two** numbers: similar digits land close together
- Pick a point where the sevens live and **decode** it: a new seven. Close codes mean similar things
- A point in the latent space, the vector of numbers for one example, is its **embedding**

Directions mean something





king − man + woman ≈ queen

the same offset (a “gender” direction)
links both pairs

≈: queen is the nearest word vector
after excluding king, man and woman

word2vec, Mikolov et al. (2013); schematic 2-D picture

- Faces with glasses − faces without = a “glasses” direction. Add it and the glasses appear; the same for a **smile**
- Words: king − queen = man − woman, so king − man + woman ≈ queen. Each word is a **word vector** (a word embedding)

Faces: Radford, Metz & Chintala, arXiv:1511.06434, Fig. 7

Making new data



auto-regressive

one piece at a time, each given the ones before:

$$\prod_i p(x_i \mid x_{<i})$$

VAE

encode to z, decode back; generate from $z \sim N(0, I)$

GAN

generator vs. discriminator: a two-player game

diffusion

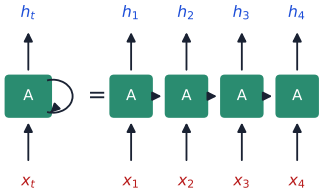
learn to remove noise step by step; start from pure noise

normalizing flow

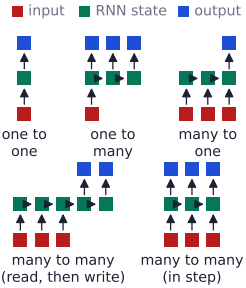
invertible map from $N(0, I)$; exact $p(x)$

- Learn where real data lives, then **sample** there: there's always randomness
- A **generative model** learns to make new examples. Three kinds: a **variational autoencoder** (codes forced into a standard normal, then sampled); a **GAN** (a maker trained against a detector); **diffusion** (learn to **undo noise**)

Sequences: read, remember, decide

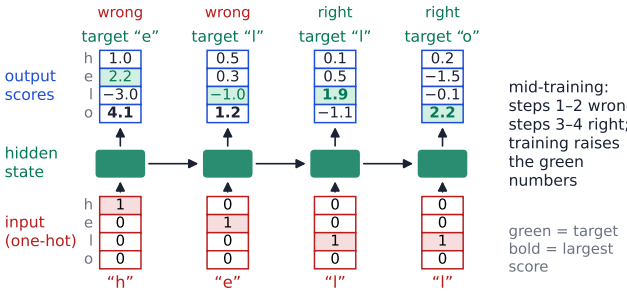


same cell A, same weights, every step



- Text and audio come in any length. Read one step at a time
- A **recurrent neural network (RNN)**: a second output, the **hidden state**, is fed back in with the next input: the **memory**
- Many-to-one: a review, good or bad. Many-to-many: translation

Predict the next token

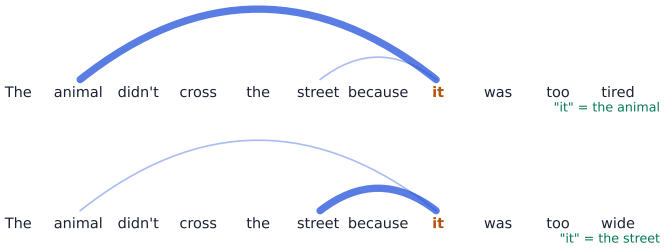


- Inputs “h e l l”, targets “e l l o”: the **labels are free**. Text is split into **tokens** (words or pieces of words)
- Do it on all the text there is: that’s how **large language models (LLMs)** are **pre-trained** (trained first on a general task, before any specific one)

Why transformers won

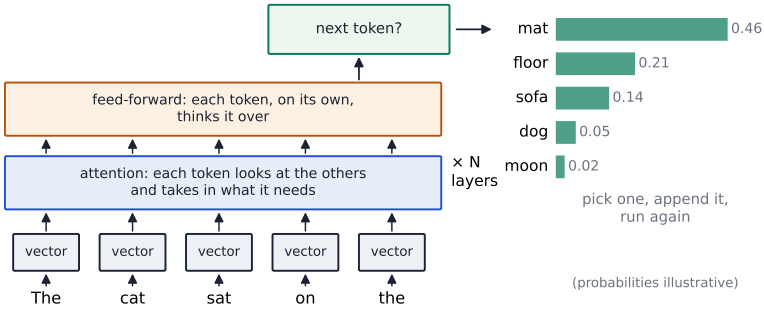
- A recurrent network can't compute step 5 before step 4: training is **sequential**
- The **transformer** drops the recurrence, so a whole sequence trains **in parallel** (all positions computed at once)
- Then scale: bigger models and more data have kept getting better

Attention: which words matter to me?



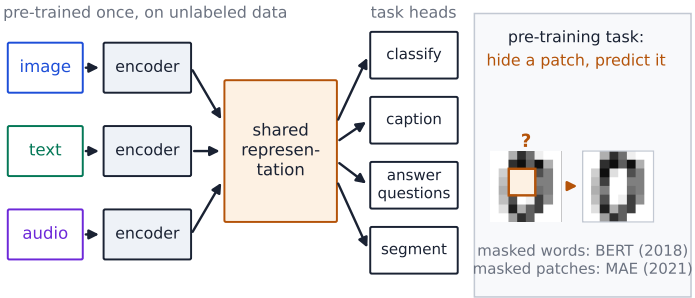
- Each word **compares** itself with every other word, turns the scores into weights, and takes a weighted mix
- What "it" means depends on the rest of the sentence: **attention** finds the words that decide it. The weights are **attention weights**, made to add up to 1 by a **softmax**

A transformer



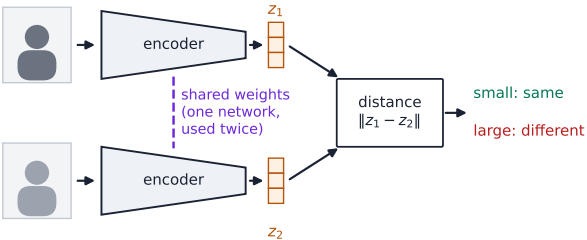
- **Attention:** tokens exchange information. **Feed-forward** layer: an ordinary dense network applied to each token on its own. Repeat
- At the top: probabilities for the next token. Pick one, append it, run again

Foundation models



- A **foundation model** is pre-trained on huge **unlabeled** data (hide part of the input, predict it), then adapted to many tasks
- One shared representation, a small head per task. They do tasks nobody trained them for

Lego blocks



- Images → convolutions. Sequences → recurrent nets or transformers. Irregular data → graphs
- The badge reader: **one** encoder on both photos, then "same person?": a **Siamese network**. Irregular data, like detector cells, uses a **graph neural network**

In my field

- 2014: deep networks on low-level detector data match decades of hand-built features
- Graph networks on calorimeters; transformers on sets of particles
- Autoencoders in the **trigger** (the system that decides, in real time, which collisions to keep), for **anomaly detection**: flag what doesn't look normal
- Networks made **blind** to mass by an **adversary** (a second network that tries to recover it), so a bump is a particle, not something the network made

Vocabulary (1)

- **feature:** a number describing an example. **model:** maps features to an answer. **label:** the known answer
- **neuron / unit:** weighted sum of inputs, then an **activation function** (sigmoid, **ReLU**)
- **weights, biases = parameters:** what training adjusts. **layer, hidden layer, deep / shallow network**
- **cost (loss) function:** how wrong the model is. **gradient:** how the cost changes with each weight
- **gradient descent, learning rate, stochastic gradient descent, mini-batch, epoch**
- **vanishing gradient:** early layers' gradients shrink to nothing, so they don't learn
- **overfitting:** memorizing the training data. **generalize:** work on new data. **validation set, learning curve, dropout**
- **representation:** a new set of numbers describing the data. **manifold:** the low-dimensional sheet the data lives on. **dimensionality reduction**
- **feature engineering** (by hand) vs **feature learning** (by the network). **feature hierarchy**

Vocabulary (2)

- **convolution, filter (kernel), feature map, pooling, convolutional neural network (CNN)**
- **activation:** the number a unit outputs. **top-5 error**
- **transfer learning:** reuse a network trained on one task for another. **pre-trained model, head, fine-tuning**
- **autoencoder: encoder** + **bottleneck** + **decoder**. **code, latent space, embedding, word vector**
- **unsupervised / self-supervised:** learning without labels
- **generative model:** makes new examples. **variational autoencoder, GAN, diffusion**
- **recurrent neural network (RNN), hidden state, token, large language model (LLM), pre-training**
- **transformer, attention, attention weights, softmax, feed-forward layer, foundation model**
- **Siamese network, graph neural network, trigger, anomaly detection, adversary**

What comes next

- **scikit-learn:** classical machine learning in a few lines (Lecture 19)
- **The software:** how frameworks compute and differentiate, and Keras (Lecture 21)
- **The pieces:** layers, activations, costs, optimizers, over-fitting, and the architectures